The E-Commerce Platform

Business & Technical Document

Using AWS

Created by
Sangeetha Vedavyasalu

Version	Version History	Modified By	Date
1.0	Initial version	Sangeetha Vedavyasalu	04/18/2024

Table of Contents

Executive Summary

This report outlines business objectives, use cases, and the proposed design and implementation strategy for developing a scalable, secure, and cloud-native e-commerce platform using Amazon Web Services (AWS). The solution focuses on meeting key functional requirements and non-functional requirements.

Business Objectives

- Build a user-friendly, responsive online store
- Support high traffic during seasonal sales
- Ensure secure user and transaction data
- Enable fast time-to-market and easy scalability
- Support global users with low-latency access

Functional Requirements

- Secure user authentication (SSO, MFA)
- Product catalog, shopping cart, and checkout
- Multiple payment gateway integrations with failover
- Order tracking, notifications, and user reviews
- Admin dashboard with analytics and role-based access

Non-Functional Requirements

- 99.99% uptime, auto-scaling for peak loads
- API response time < 200ms, payment success rate > 98%
- PCI DSS, GDPR, and CCPA compliance
- Real-time monitoring and graceful degradation strategies
- Globalization

Detailed Functional Requirements

1. **User Management**

- Allow users to **register, login, and manage accounts**
- Support **role-based access control** (e.g., customer, admin, seller)
- Enable **multi-factor authentication (MFA)** and password recovery

2. **Product Catalog**

- Manage product listings with **name, description, images, price, stock**
- Organize products into **categories, tags, and collections**
- Support **product variants** (e.g., size, color)
- Search and filter products based on attributes

3. **Shopping Cart & Wishlist**

- Enable users to **add/remove/update** items in a cart
- Store cart data for **guest and logged-in users**
- Allow users to **save items to a wishlist** for later

4. **Checkout & Orders**

- Provide a **secure checkout process** with shipping and payment steps
- Collect and validate **billing and shipping addresses**
- Support **guest checkout**
- Generate and manage **order IDs, order history, and tracking**

5. **Payment Integration**

- Integrate with **multiple payment gateways** (e.g., Stripe, PayPal)
- Handle **credit/debit cards, wallets, bank transfers**
- Support **secure payment processing** (PCI-DSS compliance)
- Provide **payment confirmation and failure handling**

6. **Shipping & Delivery**

- Offer **real-time shipping rates** from carriers
- Allow users to **select delivery options**
- Provide **order tracking** and status updates

7. **Product Reviews & Ratings**

- Allow users to submit **product ratings and written reviews**
- Moderate reviews for spam or abuse
- Show **average ratings** on product pages

8. **Notifications & Communication**

- Send **email/SMS/push notifications** for order events
- Enable **newsletter subscriptions**

- Provide **contact forms** and support chat integration

9. Search & Recommendations

- Enable **keyword search** with autocomplete
- Provide **faceted filters** (e.g., price, brand, rating)
- Use **recommendation engine** (e.g., related or trending products)

10. Promotions & Discounts

- Support **promo codes, coupons, and time-based offers**
- Implement **bulk or conditional discounts**
- Track **promotion usage and expiration**

11. Order Management (Admin)

- View and manage orders by status (e.g., pending, shipped, refunded)
- Update order fulfilment and generate invoices
- Process **returns, cancellations, and refunds**

12. Inventory Management

- Track **stock levels**, auto-restock alerts
- Support **warehouse or multi-location inventory**
- Prevent overselling via **inventory reservation**

13. Analytics & Reporting

- Generate reports on **sales, inventory, user behavior**
- Visual dashboards for performance monitoring
- Export reports in CSV/PDF formats

14. Content Management System (CMS)

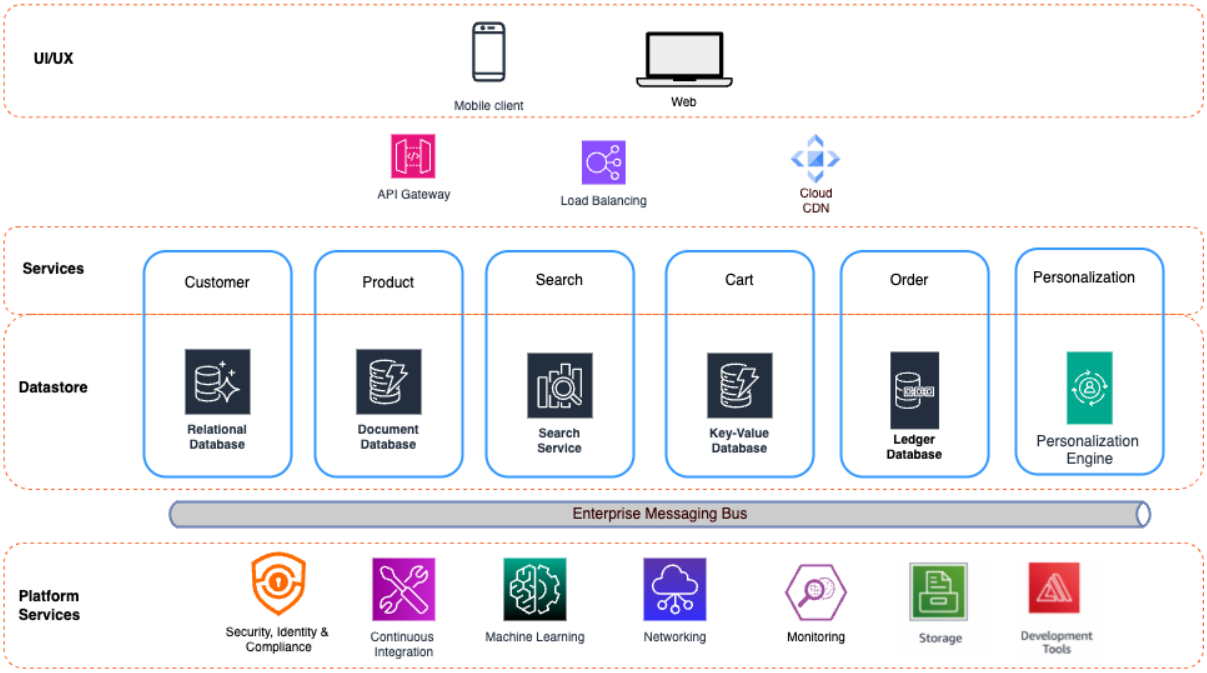
- Manage **static content** like About, FAQs, Blog
- Upload **banners, homepage features, and promotions**

15. Multi-Language & Multi-Currency Support

- Translate UI and product info into **multiple languages**
- Allow **currency selection** with real-time conversion

Architectural Overview

High-Level Architecture Diagram



Core Services Overview

Component	AWS Service(s)
Frontend Hosting	Amazon S3 + CloudFront
API Gateway	Amazon API Gateway
Business Logic	AWS Lambda / Amazon ECS (Fargate)
Authentication	Amazon Cognito
Database	Amazon RDS (PostgreSQL), DynamoDB (for NoSQL), Ledger Database
Search	Amazon OpenSearch
File Storage	Amazon S3
Caching	Amazon ElastiCache (Redis)
Messaging	Amazon SQS, SNS
Notifications	Amazon SES (Email), Amazon Pinpoint (SMS/Push)

Monitoring	Amazon CloudWatch, AWS X-Ray
Security	AWS WAF, Shield, IAM, KMS
CI/CD	AWS CodePipeline, CodeBuild, CodeDeploy

Architecture Components

1. **Frontend (Web + Mobile)**
    - React/Next.js or Angular hosted on **Amazon S3**
    - **Amazon CloudFront** for global delivery
    - Uses **HTTPS with ACM** (SSL/TLS certificates)
2. **API Layer**
    - RESTful or GraphQL APIs via **Amazon API Gateway**
    - Connects to **AWS Lambda** or **ECS (Fargate)** services
3. **Authentication**
    - **Amazon Cognito** for user pools and identity federation
    - Supports OAuth2/SAML/Google/Facebook login
4. **Business Logic**
    - Stateless microservices using **Lambda** (preferred) or **ECS**
    - One microservice per domain: user, product, cart, order, payment
5. **Data Layer**
    - **Amazon RDS** for relational data (customers, orders, transactions)
    - **DynamoDB** for product catalog and shopping cart (low-latency)
    - **Amazon S3** for product images, receipts
6. **Search & Recommendations**
    - Full-text search using **Amazon OpenSearch**
    - Optional: ML-powered recommendations using **Amazon Personalize**
7. **Order & Event Processing**
    - **Amazon SQS** for decoupling order pipeline
    - **AWS Step Functions** for order workflows (order → payment → fulfillment)
8. **Notifications**
    - **Amazon SES** for order emails

- **Amazon SNS / Pinpoint** for mobile push and SMS

9. **Monitoring & Logging**

    - **Amazon CloudWatch** for logs, alarms, metrics
    - **AWS X-Ray** for tracing across distributed services
    - **CloudTrail** for auditing API calls

10. **Security**

    - **IAM** for access control
    - **AWS WAF** for blocking malicious traffic
    - **AWS Shield** for DDoS protection
    - **AWS KMS** for encryption keys

Data Flow

- **User Request**: Client sends HTTP request via CloudFront.
- **API Layer**: Routes to API Gateway, which triggers Lambda functions or microservices.
- **Business Logic**: Services process data, interact with database, apply discounts/rules.
- **Data Access**: Reads/writes to RDS or DynamoDB.
- **Event Handling**: Events are published to SQS/SNS for background processing.
- **Response**: Result is returned to the client via API Gateway and CloudFront.

Detailed Data Flows

1. **User Accesses the Website**

    - **Scenario:** User opens the e-commerce website to browse products.
    - **Flow:**
        - **DNS Resolution**:
            - User types `www.mystore.com`
            - DNS (Amazon Route 53) resolves to **CloudFront CDN**
        - **Content Delivery**:
            - Static assets (HTML, JS, CSS, images) are served from **CloudFront + S3**
            - React/Next.js frontend is hydrated in browser

2. **Product Browsing and Search**

    - **Scenario**: User searches for “wireless headphones”.
    - **Flow:**
        - **Frontend**:
            - Search input triggers an API call to `/api/search?query=wireless+headphones`
        - **API Gateway**:

        - Receives request, routes it to a **Lambda function** or **Search Service (ECS/EKS)**
    - **Search Service**:
        - Query is run against:
            - **Elasticsearch/OpenSearch** for full-text product search
            - **DynamoDB** or **RDS** for structured data (filters, price, availability)
    - **Response**:
        - Aggregated search results are returned to frontend as JSON
        - UI renders product cards

3. Add to Cart

- **Scenario**: User adds an item to their cart.
- **Flow:**
    - **Frontend**:
        - Sends a POST request to `/api/cart/add` with product ID, quantity, user token
    - **Authentication Layer**:
        - Token is verified via **Amazon Cognito**
    - **Cart Service**:
        - Looks up product availability
        - Updates user's cart in **DynamoDB (cart table)**
    - **Response**:
        - Confirms cart update
        - UI reflects new cart state

4. Checkout and Payment

- **Scenario**: User proceeds to checkout and completes payment.
- **Flow:**
    - **Frontend**:
        - Displays cart summary
        - Collects shipping info
        - Tokenizes card via **Stripe.js** (or another payment SDK)
    - **API Request**:
        - /`api/checkout` is called with cart details, user info, and tokenized payment data
    - **Checkout Service**:
        - Authenticates user
        - Verifies cart and inventory
        - Initiates **payment with Stripe/PayPal API**
        - On success:
            - Creates **order record in RDS (Order table)**
            - Triggers stock updates via SQS message to Inventory Service
    - **Response**:
        - Payment success, order confirmed
        - Confirmation email via SNS trigger or external email service (e.g., SES or SendGrid)

5. **Order Fulfilment**

- **Scenario**: Admin or warehouse receives the order.
- **Flow:**
  - **Order Event Triggered**:
    - EventBridge or SNS notifies fulfilment system of a new order
  - **Inventory Update**:
    - Inventory Service consumes SQS message
    - Deducts stock in **DynamoDB or RDS inventory table**
  - **Shipping Label Creation**:
    - Integration with 3rd-party shipping API (e.g., Shippo, EasyPost)
    - Tracking number is saved to order record
  - **Notification**:
    - Customer receives SMS/email with shipping confirmation

6. **User Views Order History**

- **Scenario**: User logs in and checks past orders.
- **Flow:**
  - **Frontend**:
    - User authenticates via Cognito
    - Sends request to /api/orders
  - **Order Service**:
    - Queries RDS for orders associated with user ID
    - Returns list with status, items, total, tracking info
  - **UI**:
    - Displays order cards with re-order option, delivery status, etc.

7. **Background Processes (Asynchronous)**

- **Email/SMS Notifications**:
  - Order status changes → EventBridge → SNS or Lambda → SES or external service
- **Stock Replenishment Alerts**:
  - Inventory below threshold → Alert to admin or auto-purchase
- **Analytics Pipeline**:
  - Events sent to **Kinesis / Firehose** for downstream BI (e.g., Redshift, QuickSight)
- **Logs & Metrics**:
  - All API calls and backend processing are logged in **CloudWatch**
  - Performance monitored with **X-Ray**

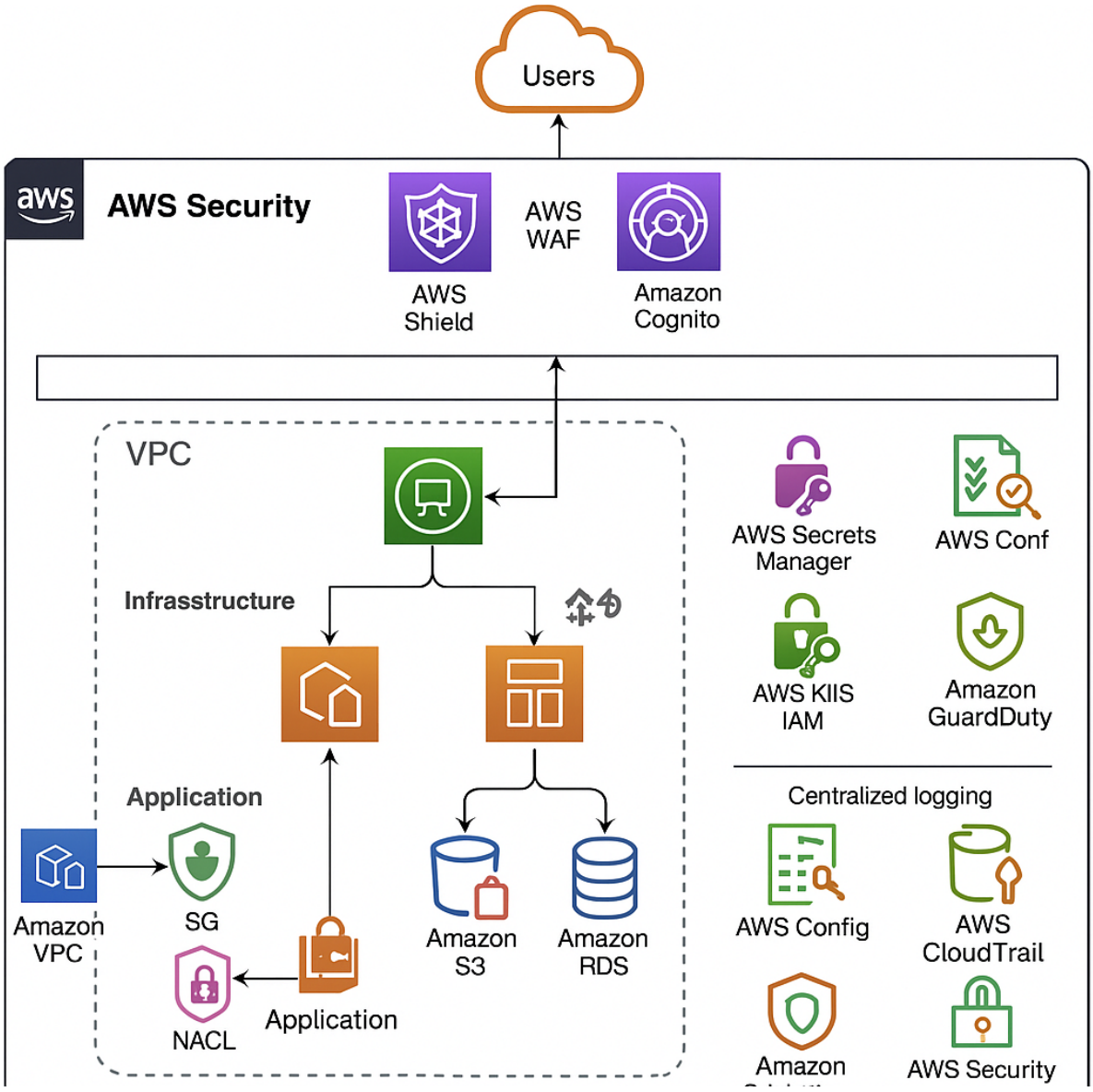
Frequency, Volume & Latency

- **Volume** refers to payload size (not file size of images/assets)
- **Latency requirement** is **user-perceived delay** (frontend/backend combined)
- **Total daily volume** reflects approximate use in a **medium-sized e-commerce platform** with ~50K–100K daily active users

- **Search & Recommender** systems often generate the highest read traffic

Functional Area	Frequency	Volume per Request	Latency Requirement	Total Daily Volume (Est.)
User Management	1–5 ops/user/day	~1–10 KB	< 500 ms	50–100K requests/day
Product Catalog	10–50 reads/user/day	2–20 KB/item	< 300 ms (read)	1M–10M requests/day
Cart & Wishlist	5–20 ops/user/session	0.5–2 KB	< 200 ms	100K–500K requests/day
Checkout & Orders	1–2 transactions/user/month	5–50 KB/order	< 500 ms	10K–50K requests/day
Payments	1 per order	1–5 KB	< 200 ms (critical)	10K–50K requests/day
Shipping & Delivery	1–3 ops/order	1–3 KB (tracking info)	< 1 sec (API dependent)	30K–100K external API calls/day
Reviews & Ratings	1–5 reviews/user/month	0.5–2 KB	~1 sec (async OK)	10K–100K submissions/month
Search & Recommender	5–20 queries/user/session	~0.5–3 KB/query	< 100–200 ms	1M–5M queries/day
Promotions & Discounts	1–3 validations/session	~0.5 KB	< 200 ms	100K–500K validations/day
Order Management (Admin)	10–100 updates/day	5–20 KB	< 1 sec	1K–10K admin ops/day
Inventory Management	1 update/order or cart change	0.5–1 KB	< 200 ms	100K–300K updates/day
Analytics & Reporting	Daily/Hourly batch jobs	MB–GB range	Low	1–10 GB/day
Notifications	2–10 notifications/user/order	0.5–2 KB	~1–5 seconds (async)	500K–2M messages/day
CMS (Content Pages)	1–5 page loads/user/session	10–100 KB (cached)	< 300 ms (via CDN)	500K–2M page loads/day
Multi-language/Currency	Depends on user geography	~0.5 KB/request	< 300 ms (API dependent)	200K–1M lookups/day

Security



Core Security Principles

Principle	Implementation in AWS
Least Privilege	Fine-grained IAM roles & policies
Defense in Depth	Layered controls at every stack layer
Zero Trust	Identity-centric access with no implicit trust
Encryption Everywhere	TLS in transit + KMS for data at rest
Audit Everything	Centralized logging and event tracking

Identity and Access Management (IAM)

Components Used:

- **IAM Users, Roles, and Policies**
- **Amazon Cognito** (for customers)
- **IAM Identity Center (SSO)** (for staff/admin access)

Best Practices:

- Use **IAM Roles with strict trust policies** for all compute services (Lambda, EC2, ECS).
- Separate **customer identity management** using **Amazon Cognito**:
  - OAuth2, SAML, OpenID Connect supported
  - Multi-Factor Authentication (MFA)
- Use **resource-based policies** for S3, Lambda, API Gateway.

Network Security

VPC Design:

- **Public Subnets**: Load balancers, NAT gateways
- **Private Subnets**: App servers, RDS, Redis
- **No internet access** for private subnets (only via NAT Gateway)

Security Controls:

- **Security Groups**: Stateful firewall at instance level (only allow specific port/IP traffic)
- **Network ACLs**: Stateless firewall for subnet-level control
- **VPC Peering / Transit Gateway**: For microservices separation
- **AWS PrivateLink**: Secure, private access to AWS services and 3rd-party APIs

Application & API Protection

AWS WAF (Web Application Firewall):

- Protects against OWASP Top 10
- Rate limiting, geo-blocking, IP filtering
- Custom rules for API abuse protection

AWS Shield:

- **Standard**: DDoS protection for all AWS users
- **Advanced**: For enhanced protection + incident response team (for business-critical platforms)

API Gateway Security:

- OAuth2 / Cognito integration
- Throttling, API keys, usage plans
- Request validation and schema enforcement

Data Protection

In Transit:

- HTTPS/TLS 1.2+ for all traffic (enforced via ALBs, CloudFront)
- Encrypted connections to databases and third-party APIs

At Rest:

- **S3**: Server-Side Encryption (SSE-S3, SSE-KMS)
- **RDS, DynamoDB, EBS**: Encrypted using **KMS Customer Managed Keys**
- **Secrets Manager / Parameter Store**: For storing database creds, API keys

Key Management

AWS KMS with:

- Customer Managed Keys (CMKs) with auto-rotation
- Access logging for key usage
- Tight IAM policies restricting who can use keys

Infrastructure Security

CI/CD Security

- Code signing & verification
- IAM role separation between build and deploy
- Secrets injection via Secrets Manager (never in code or ENV)

Container Security (if ECS/EKS used)

- Image scanning (Amazon ECR + Inspector)
- Least privilege task roles
- Use **Fargate** for isolation benefits

Security Automation & Threat Response

- **EventBridge + Lambda**:
  - Automatically respond to GuardDuty findings
  - Isolate compromised instances, revoke roles, notify security team

- **AWS Macie**:
    - Discover and classify sensitive data in S3 (PII, PHI, etc.)

Monitoring, Logging, and Auditing

CloudTrail:

- Captures all API activity (who, what, when, where)
- Stores logs securely in an S3 bucket with versioning

AWS Config:

- Tracks resource changes and compliance drift

CloudWatch:

- Logs from Lambda, EC2, ECS, and custom apps
- Metrics and alarms (e.g., login failures, throttling, latency)

GuardDuty:

- ML-based threat detection
- Flags anomalous activity, credential misuse, or potential compromise

Security Hub:

- Centralized view of all findings across GuardDuty, Config, IAM Access Analyzer, etc.
- Maps to CIS Benchmarks and PCI DSS

- **X-Ray** (trace service calls and latency)
- **Elastic Load Balancer (ELB) metrics**

Compliance and Data Governance

PCI-DSS: For handling credit card transactions (via Stripe/PayPal → offload PCI scope)

GDPR: Data protection, right to be forgotten, consent tracking

AWS Artifact: Provides reports and evidence for audits (SOC, ISO, etc.)

Use **data tagging** and **classification** for identifying and controlling sensitive data

DevOps & Deployment

Source Control: GitHub / CodeCommit

CI/CD Pipeline: AWS CodePipeline + CodeBuild

Infrastructure as Code: AWS CloudFormation / Terraform

Automated Testing: Unit and integration tests in CodeBuild

Risks and Mitigation

Failure Points & Fault Tolerance by Use Case

Functional Area	Potential Failure Points	Fault Tolerance Mechanisms
User Management	Auth service down, DB access error, rate limit hits	Multi-AZ user pool (e.g. Cognito), retries, fallback login cache
Product Catalog	DB/query timeouts, service unresponsive, cache miss	Read replicas, global cache (Redis/Memcached), circuit breakers
Cart & Wishlist	Session store failure, API throttling, DB write failure	Auto-scaled APIs, sticky sessions with fallback, DynamoDB TTL
Checkout & Orders	Order service failure, DB write issue, partial failures	Idempotent order APIs, distributed transactions, queue buffering
Payments	Third-party gateway unavailability, timeout, duplicate txn	Retry with backoff, idempotency tokens, failover gateways
Shipping & Delivery	External API failure, delayed response, bad data formats	API proxy caching, timeout fallbacks, retry with exponential backoff
Reviews & Ratings	Moderation queue delay, DB lock, spam detection glitch	Async queues (e.g., SQS), dead-letter queues, backup writer
Search & Recommender	Index not updated, high latency, recommender model down	Replicated search clusters, cache layer, fallback to generic results
Promotions & Discounts	Expired promo cache, race conditions, rule engine errors	Promo validation cache, auto-expiry cache, validation retries
Order Management (Admin)	Dashboard fails, delayed write, stale data	Dashboard mirrors, async updates, snapshot restore

Inventory Management	Over-selling due to sync issues, DB latency	Atomic operations, real-time cache + DB sync, stock buffer window
Analytics & Reporting	ETL failures, batch delays, storage outages	Job retries, partitioned processing, S3 versioning & backups
Notifications	SNS/SQS drop, email/SMS API failure	DLQs (Dead Letter Queues), backup channels (SMS/email), retries
CMS (Content Pages)	CMS downtime, slow rendering, CDN cache expired	CDN fallback, statically generated pages, cache warming scripts
Multi-lang/Currency	External translation/FX API delay, unsupported locale	Localized fallback, cache popular currency rates, polyglot fallback

Contingency Plans by Layer and Function

System Component	Failure Scenario	Contingency Plan
User Authentication	Auth service/Cognito down	Enable backup identity provider (e.g. social login); local session cache fallback
Database (Product/User/Order)	DB write/read errors, regional outage	Multi-AZ deployment, automated failover, point-in-time recovery (PITR), RPO <5 min
Payment Gateway	Third-party failure, transaction timeout	Retry with exponential backoff, alternate payment provider integration
Order Processing	Queue/delivery failure, partial transaction	Use message queues (SQS), store order in pending state, dead-letter queue (DLQ)
Search Engine	Search index corruption, engine crash	Fallback to cached results or category-based navigation
Inventory Management	Oversell due to update failure or lag	Use inventory buffer, eventually consistent updates, alert on stock anomaly

Checkout Flow	One service in chain fails (e.g., pricing service)	Use cached pricing or backup microservice version
Shipping API	3rd-party unavailable	Retry mechanism, switch to backup courier APIs, notify customer of delay
CMS (Content Delivery)	CMS service down or timeout	Serve static cached version via CDN (CloudFront/S3)
Promotions Engine	Fails to apply or validate	Default to standard discount, notify ops/admin dashboards
Notifications System	SNS/SQS or email/SMS provider failure	Retry via backup channel, use queue persistence, DLQ
File Storage (images etc.)	S3 unavailable	Use multi-region replication, CDN cache fallback (CloudFront)
Analytics & Reporting	Data pipeline failure	Retry batch jobs, use raw log backups (S3), switch to delayed reporting
CI/CD Pipeline	Deployment failure, stuck release	Automated rollback, blue/green or canary deployment model
Multi-region Failover	Region outage (e.g., AWS us-east-1 goes down)	Route 53 failover policy, deploy in multiple AWS regions

Key Cross-Cutting Contingency Strategies

Strategy	Description
Backup & Restore	Regular automated backups of DB, app config, S3 assets
Disaster Recovery Plan (DRP)	Documented recovery steps for infra/app, tested quarterly
Failover Routing	Use Route 53 health checks + weighted/routing policies
Health Checks & Auto Healing	ALBs, EC2/ASG, Lambda – auto restart unhealthy resources
Data Durability SLAs	Use S3 (11 9s durability), DynamoDB with global tables

Canary Deployments	Minimize user impact during feature rollout failures
Chaos Engineering	Intentionally introduce faults (e.g., with AWS Fault Injection Simulator)

Recovery Time Objectives (RTO) & Recovery Point Objectives (RPO)

Component	RTO Target	RPO Target
Auth & Login	< 2 mins	~0 (stateless)
Product Catalog	< 5 mins	< 15 mins
Orders & Payments	< 2 mins	< 5 mins
File Assets (Images)	< 10 mins	< 1 hr
Inventory System	< 5 mins	< 15 mins
Analytics & Reports	< 2 hrs	< 24 hrs

Performance Objectives

System Behavior: Average Load vs Peak Load

Aspect	Average Load Behavior	Peak Load Behavior (e.g., sale events, holidays)
User Traffic	Moderate, predictable traffic	Sudden spikes (5x–20x), unpredictable access patterns
Auto-scaling	Minimal or no scaling needed, fixed capacity often sufficient	Aggressive auto-scaling of compute and services (Lambda, ECS, ASG)
Latency Expectations	<200–500ms response time across services	Maintain <1 sec or fallback gracefully under pressure
Caching Use	Standard usage for products, sessions, search	Extensive use of CDN, Redis, local in-memory caches
Database Behavior	Normal read/write throughput	Read-heavy load → rely on read replicas or DynamoDB auto-scaling

Queue Systems (SQS, Kafka)	Low-to-moderate message rates	Bursts of messages, buffer requests for async processing
Search & Recommender	Normal personalized responses	Fallback to precomputed or cached results if latency spikes
Inventory Management	Real-time updates with minimal contention	Use optimistic concurrency and pre-allocated stock (buffering)
Checkout & Payments	Normal flow with full checks and verifications	May switch to partial order logging or deferred payment validation
Email/SMS Notifications	Real-time processing	Queue-based, bulk throttling via SNS/SQS/SES
Error Handling	Show detailed errors	Show generic errors or queue requests silently to reduce UX impact
Monitoring & Alerts	Standard health checks and threshold alerts	Real-time dashboards, anomaly detection, and incident response
CDN (Content Delivery)	60–80% cache hit ratio	Aim for 95%+ cache hits to reduce origin server load
Cost Control Measures	Optimize for cost/performance	Accept higher short-term cost for uptime/performance

Key Technical Adjustments During Peak Load

Adjustment	**Purpose**
Pre-warming Auto Scaling Groups	Avoid cold starts for EC2/ECS
Enable AWS WAF & Throttling	Block bad traffic and bots
Provisioned Throughput (DynamoDB)	Set max read/write ahead of time to handle burst
Use of Feature Flags	Turn off non-critical services under load (e.g., reviews)
Static Rendering	Convert dynamic content (e.g., CMS) into static HTML
Priority Queuing	Prioritize orders over non-critical notifications
Async Operations	Offload non-critical tasks to queues (e.g., emails, logs)

Example: Black Friday Peak

- **Avg Load**: 2K concurrent users → 100 req/sec
- **Peak Load**: 100K concurrent users → 5K req/sec
- **Strategy**: Scale out 50+ EC2/Lambda workers, cache 90% product pages, use async order pipeline, increase DynamoDB RCU/WCU 20x temporarily

Load Testing

1. **Objectives**
   - **Performance Benchmarking** - Establish system response under expected and extreme loads
   - **Scalability Validation** - Test auto-scaling triggers, bottlenecks, and elasticity
   - **Capacity Planning** - Estimate infrastructure needed for peak load (e.g. Black Friday)
   - **Resilience Verification -** Ensure fault tolerance and graceful degradation under stress

2. **Types of Load Tests**

   Smoke Test - Light load to verify if the system is up and responsive

   Baseline Test - Establish metrics under average load (e.g., 1K users)

   Stress Test - Push system beyond max capacity to find breaking point

   Spike Test - Sudden burst of traffic (e.g., flash sale trigger)

   Soak Test - Sustained load for hours to test memory leaks or performance drift

3. **User Scenarios to Simulate**

Scenario	% of Users	Requests/User	Notes
Home Page Browsing	30%	3–5	CDN-backed
Product Search & Catalog View	25%	5–10	High read frequency
Add to Cart / Wishlist	15%	2–5	Writes + session
Checkout Flow	10%	1–2	Includes auth, inventory, payment
User Login / Signup	10%	1–2	Cognito or auth backend

Submit Review / Rate Product	5%	1	Optional
Admin Inventory Update	5%	5	Simulated at lower freq

4. **Key Metrics to Capture**

Metric	Expectation
Response Time (p50/p95/p99)	Should be < 200ms for most endpoints
Error Rate (%)	Should be < 1%
Throughput (req/sec)	Measure peak handling capacity
CPU/Memory/Network Usage	Monitor saturation points
Auto-scaling Events	Ensure ASG, Lambda, DB scale appropriately
DB Throughput	Monitor RCU/WCU or read/write IOPS
Queue Depth (SQS/Kafka)	Ensure no backlog builds during load

5. **Tools to Use**

JMeter - GUI, high configurability

Locust - Python-based, simulates user behavior

Artillery - Lightweight Node.js-based, good for CI/CD

Gatling - Scala-based, good for long-running tests

Scalability

Scalability is the ability of a system to **handle increasing workloads or user demands** by either **adding more resources** (scaling out/up) or **optimizing existing ones** without sacrificing performance or availability.

Key Attributes of Scalability

- **Elasticity**: Dynamically adjusts to load (both up and down)
- **High availability**: Works even during heavy load
- **Performance consistency**: Maintains response times under stress
- **Cost-efficiency**: Scales only when needed

Scalability using AWS

1. **Stateless Microservices Architecture**

   - Decouple components like cart, checkout, search, etc.
   - Deploy independently with AWS Lambda, ECS, or EKS

2. **Use Auto Scaling & Load Balancers**

Component	AWS Service	Scaling Mechanism
App Servers	EC2 + ASG	Auto Scaling Group + ELB
APIs	Lambda	Concurrency-based scaling
Containers	ECS / EKS	Fargate or Cluster Auto Scaling
Caching	ElastiCache	Redis/Memcached clusters
DB Layer	Aurora / DynamoDB	Auto Scaling, Read Replicas, Global Tables

3. **Implement Asynchronous Communication**

   - Use **Amazon SQS** and **SNS** to buffer workloads and decouple services
   - Avoid bottlenecks with queue-based order processing and notifications

4. **Edge Delivery with CDN**

   - Use **CloudFront** to serve static/dynamic content globally with low latency

5. **Database Partitioning & Caching**

   - **DynamoDB** with Global Tables for write-heavy and global scale
   - **Aurora Read Replicas** for read-heavy workloads
   - Add **ElastiCache (Redis)** for frequent reads (product catalog, cart)

6. **Event-Driven & Serverless**

   - Use **EventBridge**, **Step Functions**, **Lambda** for event-driven workflows
   - Serverless scales automatically and costs only on usage

7. **Monitoring & Auto Scaling Triggers**

   - Use **CloudWatch** for custom metrics: CPU, latency, queue depth
   - Setup alarms and scale policies for critical thresholds

Example Use Case: Flash Sale

Event	Challenge	Scaling Strategy
100x traffic	Sudden spike in product views	CloudFront + DynamoDB on-demand
Cart storm	Many users adding items	Lambda + DAX cache
Checkout overload	Payments & orders peak	Queue orders via SQS + async Lambda processing
Data writes	Rapid inventory changes	DynamoDB with adaptive capacity

Multi-Region and High Availability

This section focuses on a **multi-region, high-availability (HA) strategy** using AWS. This ensures **global performance**, **disaster recovery**, and **business continuity**.

Multi-Region Deployment

Main objectives

- **Minimize latency** for global users
- **Ensure fault tolerance** in case of regional AWS outages
- **Achieve disaster recovery (DR)** with automated failover
- **Comply with regional data residency or legal requirements**

Recommended Multi-Region Deployment Model

Layer	Strategy
Frontend (UI)	Deploy frontend in multiple S3 buckets + CloudFront POPs for global low-latency access
API Layer	Deploy API Gateway + Lambda in multiple regions using **Route 53 latency-based routing**
Business Logic	Duplicate stateless Lambda or container-based microservices across regions
Databases	Use **Aurora Global Database** (for RDS) and **Global Tables in DynamoDB**
File Storage	Use **S3 Cross-Region Replication** for product images and static assets
Search	Deploy **OpenSearch in active-active** or **read-only replica** regions

Messaging	Use **SQS with cross-region replication** via Lambda or EventBridge
Monitoring	Centralize logs in one region with **CloudWatch cross-account/cross-region dashboards**

High-Availability Patterns per Component

2. **API Gateway & Lambda**

    - **API Gateway** with **Lambda functions** deployed in multiple regions
    - Use **Route 53 latency-based DNS routing** to direct users to the closest endpoint

3. **Compute (ECS/EKS)**

    - **ECS with Fargate** or **EKS** clusters deployed in two or more regions
    - Use **Load Balancers** in each region
    - Global traffic routed via **Route 53** or **Global Accelerator**

4. **Databases**

    - **Relational (Aurora Global DB)**:
        - Primary in one region
        - Read replicas in one or more secondary regions
        - Failover within ~1 minute in case of region failure
    - **NoSQL (DynamoDB Global Tables):**
        - Multi-master writes in different regions
        - Ensures strong eventual consistency

5. **File Storage (S3)**

    - **S3 with Cross-Region Replication (CRR):**
        - Automatically replicates data across buckets in other regions
        - Ensure public assets (e.g., product images) are globally available

6. **Caching & Search**

    - Deploy **ElastiCache** (Redis) and **OpenSearch** in each region
    - Use **read-only OpenSearch clusters** in secondary regions if full sync is not needed

Disaster Recovery Strategy

Tier	Recovery Strategy	Recovery Time Objective (RTO)	Recovery Point Objective (RPO)
Web/App	Active-Active (instant failover)	~minutes	Near-zero
Database	Aurora Global DB / DynamoDB Global Tables	1–5 minutes	Seconds
Static Files	S3 with CRR	Immediate	Near-zero
Infra	IaC redeploy via CloudFormation/Terraform	Hours (if cold standby)	Based on latest backup

Tools to Enable Multi-Region Strategy

Category	AWS Service(s)
DNS Routing	**Amazon Route 53** (Latency-based, Failover)
Global Load Balancing	**AWS Global Accelerator** (Optional for TCP/UDP)
Data Sync	**S3 CRR**, **DynamoDB Global Tables**, **Aurora Global DB**
CI/CD Deployment	**CodePipeline with multi-region deployment scripts**
Monitoring & Alerts	**CloudWatch, CloudTrail, AWS Health Dashboard**
Security	**IAM**, **WAF**, **KMS Multi-Region Key Replication**

Global Deployment Example (Two Region Setup)

Component	Region 1 (Primary)	Region 2 (Secondary)
Frontend (S3)	us-east-1	eu-west-1
API Gateway + Lambda	us-east-1	eu-west-1
Aurora Global DB	Writer: us-east-1	Reader: eu-west-1
DynamoDB	Global Table	Global Table
S3	Primary	Replicated via CRR
OpenSearch	Primary cluster	Read-only replica

Monitoring	Centralized in us-east-1	Shared metrics/logs

Multi-Region Considerations & Trade-offs

Area	Consideration
Latency	May increase slightly if replication is delayed
Cost	Higher due to duplication of resources and cross-region traffic
Consistency	Use strong consistency where necessary (Aurora Writer, etc.)
Failover Complexity	Requires health checks, Route 53 failover policies, replication monitoring
Compliance	Ensures regional data storage policies are respected

Implementing a multi-region strategy significantly boosts platform **resilience, availability, and user experience**, especially for global markets. AWS provides robust services that support such architecture with minimal custom engineering, enabling teams to focus on delivering customer value while maintaining reliability and compliance.

What to Architect For - Balancing Business and Cost

Scalability That's Elastic, Not Expensive

Architect For	Why It Matters	Cost-Aware Solution
Auto-scaling Services	To handle traffic surges without over-provisioning	Use AWS Lambda, ECS Fargate, or Auto Scaling EC2
Tiered Storage	Not all data needs high-cost storage	Use S3 Standard, Glacier for archival, and lifecycle policies
On-demand vs Reserved	Control over compute cost	Use Reserved Instances for baseline load, spot/on-demand for bursts

Right-Sized Microservices (Not Over-Engineered)

Architect For	Why It Matters	Cost-Aware Solution
Separation of Concerns	Better performance and deployability	But avoid 100s of microservices — maintain a cost-effective boundary

Stateless Services	Easy to scale and replace	Use AWS Fargate or Lambda for non-persistent compute tasks
API Gateway Layer	Control access and routing	Avoid overusing if traffic is internal-only

Performance That Converts (UX = Revenue)

Architect For	Why It Matters	Cost-Aware Solution
Low Latency	Every 1s delay = ~7% drop in conversion	Use CDN (CloudFront), edge caching, and local DB replicas
Fast Checkout Flow	Abandoned carts cost money	Prioritize API performance and fallback paths in checkout microservices
Product Search & Recommendations	Key sales drivers	Use managed OpenSearch with query optimization, cache results

Availability Without Overkill

Architect For	Why It Matters	Cost-Aware Solution
Multi-AZ Setup	Avoid downtime from zone failures	Standard practice with minimal extra cost
Multi-Region (Selective)	Ensure global reach/resilience	Use Route 53 + read-only replicas/CDN, not full duplicate infra
RTO/RPO Aligned with SLAs	Business continuity plans	Match DR spend to business impact (e.g., full DR for payments, relaxed for reviews)

Security That's Built-In, Not Bolted On

Architect For	Why It Matters	Cost-Aware Solution
Data encryption	Trust, compliance, and regulation	Use AWS KMS (key rotation) and encrypted S3 buckets
IAM & Access Control	Prevent insider and outsider risks	Use fine-grained IAM roles; avoid excessive services per role
Bot Protection	Save bandwidth & infra	Use AWS WAF, Bot Control (cost vs traffic savings)

Observability to Reduce Long-Term Cost

Architect For	Why It Matters	Cost-Aware Solution

Logging & Tracing	Debug faster = Save dev time = Save $$$	Use CloudWatch logs with filters, or third-party centralized logging
Health Monitoring	Auto-healing reduces downtime costs	Use Route 53 health checks, CloudWatch alarms, auto-scaling triggers
Real-Time Dashboards	For proactive responses	But use data sampling to reduce ingestion cost (e.g., AWS X-Ray sampling rules)

DevOps & Automation

Architect For	Why It Matters	Cost-Aware Solution
CI/CD Pipelines	Speed up delivery, reduce errors	Use AWS CodePipeline or GitHub Actions with environment triggers
Infrastructure as Code (IaC)	Standardizes and reduces human error	Use Terraform or CloudFormation to avoid accidental overspend
Cost Monitoring	Track what's draining your budget	Use AWS Cost Explorer, Budgets, and anomaly detection tools

Project Timeline (Inception sizing)

Phase	Duration	Key Deliverables
Requirements & Design	6 weeks	Architecture diagrams, service selection
Development	12-15 weeks	Core services (Auth, Catalog, Cart, Orders, Payments)
Testing & Hardening	6 - 8 weeks	Load testing, security audits, bug fixes
Deployment	2 weeks	Production rollout, monitoring setup

Conclusion

By leveraging AWS services, this architecture supports a scalable, resilient, and secure e-commerce platform with the flexibility to grow with business needs. Using serverless and managed services helps reduce operational overhead and allows the team to focus on delivering business value.